AICL CLI TUI

Interactive Terminal Interface

User Manual

Version 1.0.0

Philippe-Antoine | June 2026

1. Introduction

The AICL TUI (Terminal User Interface) is an interactive command-line interface inspired by Claude Code. It provides a rich, intuitive terminal experience for writing, compiling, verifying, and auditing AICL programs without leaving the terminal. Unlike the basic CLI which operates on individual commands, the TUI provides a persistent, interactive environment with a file browser, code viewer, output panel, and command input, all within a single terminal window.

The TUI is built with Textual, a Python framework for building sophisticated terminal applications. It features a dark theme optimized for long coding sessions, syntax highlighting for AICL source code, and a command-driven interface that feels natural to developers familiar with modern terminal tools like Vim, Emacs, or Claude Code.

2. Installation and Launch

```
pip install aicl
pip install textual rich
```

Launch the TUI:

```
aicl tui
```

This opens the interactive terminal interface with a sidebar file browser, main editor area, output panel, and command input bar. The TUI requires a terminal that supports 256 colors or true color. Most modern terminals (iTerm2, Windows Terminal, Alacritty, Kitty, GNOME Terminal) are compatible.

3. Interface Layout

The TUI interface consists of four main areas, each serving a distinct purpose in the development workflow:

Header (top) - Shows the application title and current time.

File Explorer (left sidebar) - Directory tree for browsing and opening .aicl files. Click any .aicl file to load it into the editor. Toggle visibility with Ctrl+E.

Editor Area (center) - Displays the current AICL source code with syntax highlighting. Keywords appear in cyan, types in green, and comments in gray.

Output Panel (bottom) - Shows compilation results, verification reports, audit summaries, and command responses.

Command Input (bottom bar) - Type commands starting with : or AICL code to interact. This is your primary interface to the TUI.

4. Commands

All TUI commands start with a colon (:). Type :help at any time to see the full list. Here is the complete command reference:

Command	Description	Usage
:help	Show all available commands	:help
:open	Open an AICL file	:open path/to/file.aicl
:save	Save current file	:save
:new	Create new untitled file	:new
:compile	Compile current file	:compile [python|rust|js|go]
:verify	Verify specification quality	:verify
:audit	Audit compilation	:audit
:proof	View Proof of Origin	:proof
:explain	Explain compilation provenance	:explain
:tree	Show architecture tree	:tree
:optimize	Optimize architecture	:optimize
:run	Run self-healing runtime	:run
:sign	Cryptographically sign proof	:sign
:verify-proof	Independent proof verification	:verify-proof
:ownership	Show ownership model	:ownership
:targets	List compilation targets	:targets
:examples	Browse built-in examples	:examples
:exercise	Start an exercise	:exercise [1-5]
:clear	Clear output panel	:clear
:version	Show AICL version	:version

5. Keyboard Shortcuts

Shortcut	Action
Ctrl+O	Open file (prefills :open)
Ctrl+S	Save current file
Ctrl+K	Command palette (prefills :)
Ctrl+E	Toggle sidebar visibility
Ctrl+Q	Quit TUI
Tab	Auto-complete keywords

6. Workflow Examples

6.1 Compile and Verify

```
:open examples/02_pong.aicl
:compile
:verify
:proof
```

This opens the Pong example, compiles it to Python, verifies the specification quality, and displays the Proof of Origin. The output panel shows each step's results incrementally.

6.2 Multi-Language Compilation

```
:compile rust # Compile to Rust
:compile js # Compile to JavaScript
:compile go # Compile to Go
```

6.3 Exercises

```
:exercise # List all exercises
:exercise 1 # Load Exercise 1: Hello World
:compile # Test your solution
:verify # Check specification quality
```

7. Exercises

The TUI includes 5 progressive exercises designed to teach AICL from basic to advanced:

#	Title	Skills Practiced
1	Hello World	Goal, Layer, Validation (Level 1)
2	Risk and Recovery	Risk/Recovery pairs, error handling
3	Entities and Behaviors	Entity definitions, Behavior actions (Levels 2-3)
4	Conditions and Events	When/Then rules, On/Action handlers (Levels 4-5)
5	Full Application	Complete chat app using all 9 levels

Each exercise loads a starter template with TODO markers. Complete the TODOs and use :compile to test your solution. Use :verify to check specification quality before compiling.

8. Troubleshooting

Display issues - Ensure your terminal supports 256 colors. Try setting TERM=xterm-256color.

Import errors - Install textual and rich: pip install textual rich

File not found - Use absolute paths or run the TUI from the project directory.

Compilation errors - Use :verify to check specification quality before compiling.